

Microcontrollers Lab — ISL48

Ramaiah Institute of Technology | Dept. of Information Science & Engineering | ARM Keil MicrovisionV

PART A — ARM Assembly Language Programs

Q1a

ALP to add first 10 odd numbers. Store sum in register.

```
AREA PROG3, CODE, READONLY
ENTRY
    MOV R1, #1          ; first odd number / accumulator
    MOV R2, #9          ; loop counter (9 more to add)
    MOV R3, #1          ; current odd number
BACKK ADD R3, R3, #2      ; next odd number
    ADD R1, R1, R3       ; accumulate sum
    SUBS R2, R2, #1
    BNE BACKK
GO     B GO
END
```

Q1b

ALP to compute sum of squares of 5 numbers starting from 1. Write and use procedure SQU. Store sum in register.

```
AREA PROG5, CODE, READONLY
ENTRY
    MOV R7, #0          ; sum
    MOV R2, #1          ; current number
LOOP  BL SQU            ; call square procedure
    ADD R7, R7, R4       ; add square to sum
    ADD R2, R2, #1
    CMP R2, #6
    BNE LOOP
GO     B GO
SQU    MUL R4, R2, R2    ; R4 = R2^2
    MOV PC, LR
END
```

Q2a

ALP to add the first n even numbers. Store the result in a memory location.

```
AREA PROG6, CODE, READONLY
N      RN 1
RESULT RN 2
EVEN_NUMBER RN 3
ENTRY
    MOV N, #5
    MOV RESULT, #0
    MOV EVEN_NUMBER, #2
    MOV R4, #0x40000000
LOOP  ADD RESULT, RESULT, EVEN_NUMBER
    ADD EVEN_NUMBER, EVEN_NUMBER, #2
    SUBS N, N, #1
    BNE LOOP
    STR RESULT, [R4]
STOP  B STOP
END
```

Q2b

ALP to generate a geometric progression with a limit n. Display the results in memory.

```
AREA PROG7, CODE, READONLY
A  RN 1
D  RN 2
N  RN 3
ENTRY
    MOV A, #1
    MOV D, #2
    MOV N, #10
    MOV R5, #0x40000000
LOOP    MUL R6, A, D
        MOV A, R6
        STR A, [R5], #4
        SUBS N, N, #1
        BNE LOOP
STOP    B STOP
END
```

Q3a

ALP to count the number of zeroes and ones in a binary number.

```
AREA PROG10, CODE, READONLY
NUMBER    RN 1
NUMONES   RN 10
NUMZEROES RN 11
ENTRY
    MOV R5, #0x40000000
    LDR NUMBER, =0xA
    MOV NUMONES, #0
    MOV NUMZEROES, #0
LOOP    LSRS NUMBER, #1      ; shift right, LSB -> carry
        ADDCS NUMONES, #1    ; carry set -> bit was 1
        ADDCC NUMZEROES, #1  ; carry clear -> bit was 0
        CMP NUMBER, #0
        BNE LOOP
        STR NUMONES, [R5]
        STR NUMZEROES, [R5, #4]
STOP    B STOP
END
```

Q3b

ALP to find the average of ten 16-bit numbers stored in memory.

```
AREA PROG11, CODE, READONLY
ENTRY
    LDR R7, =TABLE
    MOV R0, #9
    LDRH R1, [R7]
BACKK  LDRH R2, [R7, #2]!
    ADD R1, R1, R2
    SUBS R0, R0, #1
    BNE BACKK
    MOV R3, #10          ; divisor = 10
    MOV R4, #0          ; quotient (average)
    MOV R5, R1
BACKK1 SUBS R5, R5, R3
    ADDPL R4, R4, #1
    BPL BACKK1
    ADDMI R5, R5, R3      ; remainder in R5
GO      B GO
TABLE  DCW 1000, 2564, 8936, 344, 5667, 908, 786, 654, 9871, 456
END
```

Q4a

ALP to find the factorial of a number.

```
AREA PROG12, CODE, READONLY
N      RN 1
FACT   RN 2
ENTRY
    MOV N, #10
    MOV FACT, #1
LOOP   MUL FACT, N, FACT
    SUBS N, N, #1
    BNE LOOP
STOP   B STOP
END
```

Q4b

ALP to generate the first n Fibonacci numbers.

```
AREA PROG13, CODE, READONLY
ENTRY
    MOV R1, #1
    LDR R2, =TABLE
    LDR R3, =NUMFIBONACCI
    LDRB R6, [R3]
    STRB R1, [R2], #1
    MOV R3, #0
    MOV R4, #0
    MOV R5, #1
    SUB R6, R6, #1
BACKK ADD R4, R3, R1
    STRB R4, [R2], #1
    MOV R3, R1
    MOV R1, R4
    ADD R5, R5, #1
    CMP R5, R6
    BLS BACKK
GO     B GO
NUMFIBONACCI DCB 0x0A
AREA NUMBER, DATA, READWRITE
TABLE SPACE 60
END
```

Q5

ALP to find the sum of digits of a number.

```
AREA PROG14, CODE, READONLY
DIVIDEND RN 1
DIVISOR RN 2
QUOTIENT RN 3
REMAINDER RN 4
RESULT RN 5
ENTRY
    LDR DIVIDEND, =12345
    MOV DIVISOR, #10
    MOV RESULT, #0
LOOP BL DIV
    ADD RESULT, REMAINDER, RESULT
    CMP QUOTIENT, #0
    MOVNE DIVIDEND, QUOTIENT
    BNE LOOP
STOP B STOP
DIV MOV QUOTIENT, #0
LOOP2 SUBS DIVIDEND, DIVIDEND, DIVISOR
    ADDPL QUOTIENT, QUOTIENT, #1
    BPL LOOP2
    ADDMI REMAINDER, DIVIDEND, DIVISOR
    BX LR
END
```

Q6

ALP to convert BCD number to binary.

```
AREA PROG15, CODE, READONLY
RADIX          RN 0
LOWERNIBBLEMASK RN 10
UPPERNIBBLEMASK RN 11
LOWERNIBBLE     RN 3
UPPERNIBBLE     RN 4
RESULT          RN 5
NUMBYTES        RN 6
BYTE            RN 2
ENTRY
    MOV RADIX, #10
    MOV LOWERNIBBLEMASK, #0x0F
    MOV UPPERNIBBLEMASK, #0xF0
    MOV RESULT, #0
    MOV NUMBYTES, #4
    LDR R1, =NUMBER
    ADD R1, R1, NUMBYTES
    SUB R1, R1, #1
LOOP    LDRB BYTE, [R1]
        SUB R1, R1, #1
        AND LOWERNIBBLE, BYTE, LOWERNIBBLEMASK
        AND UPPERNIBBLE, BYTE, UPPERNIBBLEMASK
        LSR UPPERNIBBLE, #4
        MLA RESULT, RADIX, RESULT, UPPERNIBBLE
        MLA RESULT, RADIX, RESULT, LOWERNIBBLE
        SUBS NUMBYTES, NUMBYTES, #1
        BNE LOOP
STOP    B STOP
NUMBER DCD 0x00000127
END
```

Q7

ALP to select r objects from n objects without considering order of elements — combination method (nCr).

```
AREA PROG16, CODE, READONLY
DIVIDEND RN 1  DIVISOR RN 2  QUOTIENT RN 3
REMAINDER RN 4  N RN 10  R RN 11  NDR RN 12
ENTRY
    MOV N, #6
    MOV R, #3
    LDR R5, =0X40000000
    SUB NDR, N, R
    MOV DIVIDEND, N
    BL FACT          ; compute N!
    MOV N, DIVISOR
    MOV DIVIDEND, NDR
    BL FACT          ; compute (N-R)!
    MOV DIVIDEND, N
    BL DIV           ; N! / (N-R)!
    STR QUOTIENT, [R5]
STOP B STOP
FACT  MOV DIVISOR, #1
LOOP2 MUL DIVISOR, DIVIDEND, DIVISOR
      SUBS DIVIDEND, DIVIDEND, #1
      BNE LOOP2
      BX LR
DIV   MOV QUOTIENT, #0
LOOP3 SUBS DIVIDEND, DIVIDEND, DIVISOR
      ADDPL QUOTIENT, QUOTIENT, #1
      BPL LOOP3
      ADDMI REMAINDER, DIVIDEND, DIVISOR
      BX LR
END
```

Q8

ALP to select r objects from n objects considering order of elements — permutation method (nPr).

```

AREA PROG17, CODE, READONLY
DIVIDEND RN 1  DIVISOR RN 2  QUOTIENT RN 3
REMAINDER RN 4  N RN 10  R RN 11  NDR RN 12
ENTRY
    LDR R5, =0X40000000
    MOV N, #6
    MOV R, #3
    SUB NDR, N, R
    MOV DIVIDEND, N
    BL FACT          ; compute N!
    MOV N, DIVISOR
    MOV DIVIDEND, R
    BL FACT          ; compute R!
    MOV R, DIVISOR
    MOV DIVIDEND, NDR
    BL FACT          ; compute (N-R)!
    MOV DIVIDEND, N
    MUL DIVISOR, R, DIVISOR ; R! * (N-R)!
    BL DIV           ; N! / (R! * (N-R)!)
    STR QUOTIENT, [R5]
STOP B STOP
FACT  MOV DIVISOR, #1
LOOP2 MUL DIVISOR, DIVIDEND, DIVISOR
      SUBS DIVIDEND, DIVIDEND, #1
      BNE LOOP2
      BX LR
DIV   MOV QUOTIENT, #0
LOOP3 SUBS DIVIDEND, DIVIDEND, DIVISOR
      ADDPL QUOTIENT, QUOTIENT, #1
      BPL LOOP3
      ADDMI REMAINDER, DIVIDEND, DIVISOR
      BX LR
END

```

Q10

ALP to implement Binary Search on an array of integers.

```
AREA PROG19, CODE, READONLY
ENTRY
    STORAGE EQU 0X40000000
    LDR SP, =STORAGE
    LDR R3, =STORAGE + 200
    NUM EQU 11
    SIZE EQU 1
    ADR R6, ARRAY
    MOV R1, #0
    MOV R2, #NUM - 1
    MOV R5, #17 ; search value
    STMDB R3!, {R6,R1,R2,R5, R0}
MAIN    BL FINDIT
        B MAIN
FINDIT
    STMDB SP!, {R4,R7,R8,R9,R10,R11,R12,LR}
    LDMFD R3!, {R11,R7,R8,R10, R0}
    CMP R7, R8
    BGT STOP
    ADD R9, R7, R8
    MOV R9, R9, ASR #1
    LDR R12, [R11, R9, LSL #2]
    ADD R11, R9, LSL #2
    MOV R0, R11
    ADD R4, R9, LSL #1
    CMP R12, R10
    SUBGT R8, R9, #1
    ADDLE R7, R9, #1
    LDR R11, [R3, #-4]
    STMFD R3!, {R11,R7,R8,R10, R0}
    LDMIA SP!, {R4,R7,R8,R9,R10,R11,R12,PC}
    MOV PC, LR
STOP    B STOP
ARRAY   DCD 3,6,8,12,17,22,45,67,99,2089,30001
END
```


Q11

ALP to check whether the given number is palindrome.

AREA PROG20, CODE, READONLY

ENTRY

```
        LDR R1, = 12321
        MOV R6, R1
        MOV R2, #10
        MOV R5, #0
        MOV R10, #10
LOOP    BL DIV
        MLA R5, R10, R5, R4      ; build reversed number
        CMP R3, #0
        MOVNE R1, R3
        BNE LOOP
        CMP R5, R6              ; compare reversed with original
        MOVEQ R7, #1           ; R7=1 -> palindrome
        MOVNE R7, #0           ; R7=0 -> not palindrome
STOP    B STOP
DIV     MOV R3, #0
LOOP2   SUBS R1, R1, R2
        ADDPL R3, R3, #1
        BPL LOOP2
        ADDMI R4, R1, R2
        BX LR
END
```

Q12

ALP to count the number of times a substring is repeated in the string.

AREA PROG21, CODE, READONLY

CNT RN 7

ENTRY

```
        LDR R1, =M
        LDR R2, =S
        MOV R12, R2
        MOV CNT, #0
LOOP    LDRB R3, [R1]
        LDRB R4, [R2]
        CMP R4, #0              ; end of substring?
        ADDEQ CNT, CNT, #1      ; yes -> increment count
        MOVEQ R2, R12           ; reset substring pointer
        BEQ LOOP
        CMP R3, R4
        ADDEQ R2, R2, #1        ; chars match -> advance substring ptr
        MOVNE R2, R12           ; mismatch -> reset substring ptr
        ADD R1, R1, #1
        BEQ LOOP
        CMP R3, #0              ; end of main string?
        BEQ STOP
        BNE LOOP
STOP    B STOP
M       DCB "ABCABC", 0
S       DCB "ABC", 0
END
```

QB1

C program to toggle the lowest pin of Port 0 with a delay between the two states. Observe and record the waveform using the Logic Analyzer in the Keil simulator.

```
#include <LPC214X.h>

void delay(int);

int main()
{
    IODIR0 = 0x00000001;        // P0.0 as output
    while(1){
        IOSET0 = 0x00000001;    // set P0.0 HIGH
        delay(500);
        IOCLR0 = 0x00000001;    // set P0.0 LOW
        delay(500);
    }
}

void delay(int n)
{
    int i = 0;
    for(i = 0; i < n; i++);
}
```

QB3

C program to generate a square wave using Timer0 in the interrupt mode.

```
#include <LPC214X.h>

unsigned int x = 0;

__irq void Timer0_ISR (void){
    x ^= 1;
    if(x)
        IOSET1 = 1 << 20;
    else
        IOCLR1 = 1 << 20;
    T0IR = 0x01;                // clear interrupt flag
    VICVectAddr = 0x00000000;   // acknowledge VIC
}

int main(){
    IODIR1 = 0x0FFFFFFF;
    T0MCR = 0x00000003;        // interrupt + reset on match
    T0MR0 = 0x3456FF;          // match value
    VICVectAddr4 = (unsigned)Timer0_ISR;
    VICVectCntl4 = 0x00000024;
    VICIntEnable = 0x00000010;
    T0TCR = 1;                 // start timer
    for(;;);
}
```

QB5

C program to display the string 'I LOVE ISE' in the serial window of UART1.

```
#include <LPC214X.h>

void init()
{
    PINSEL0 = 0x05;    // select UART0 pins
    U0FCR   = 0x07;    // enable & reset FIFOs
    U0LCR   = 0x83;    // 8-bit, 1 stop, no parity; DLAB=1
    U0DLL   = 0x5D;    // baud rate divisor LSB
    U0DLM   = 0x00;    // baud rate divisor MSB
}

void delay()
{
    int i;
    for(i = 0; i < 250; i++);
}

int main()
{
    unsigned char p[] = "I LOVE ISE";
    int z;
    init();
    for(z = 0; z <= 24; z++)
    {
        U0THR = p[z];
        while(!(U0LSR & 0x20)); // wait till THR empty
        delay();
    }
    while(1);
}
```